

ADAP-GNN: Adaptive property-aware graph neural network for intrusion detection in IoT networks[☆]

Mortada Termos^{a,b}, Zakariya Ghalmane^a, Mohamed-el-Amine Brahmia^a, Ahmad Fadlallah^{b,c}, Ali Jaber^b, Mourad Zghal^a

^a CESI LINEACT, UR 7527, Strasbourg, France

^b Computer Science department, Faculty of Sciences, Lebanese University, Beirut, Lebanon

^c Computer Science department, University of Sciences and Arts in Lebanon (USAL), Beirut, Lebanon

ARTICLE INFO

Keywords:

Intrusion detection
Complex networks
Graph neural networks
Intrusion detection systems
Internet of things
Cyber security

ABSTRACT

New and sophisticated attacks are threatening Internet of Things (IoT) networks, compromising the security and trustworthiness of devices. Consequently, Network Intrusion Detection Systems (NIDS) have become critical for protecting these networks, and AI-based NIDS have emerged as a promising solution. A relatively new subfield of deep learning, Graph Neural Networks (GNNs), has further advanced this field by capturing the complex relational patterns in network data. Architectures like GraphSAGE, which employs neighborhood sampling at the node level, and Graph Attention Networks (GAT), which use an attention mechanism to emphasize important nodes, have demonstrated impressive performance across various applications. However, these architectures have not yet been fully explored in the context of NIDS. Additionally, no single GNN architecture is universally optimal across all networks, as their performance depends on the specific graph properties of the network. To address these challenges, a novel framework is proposed encompassing two main aspects: (i) adaptation of the GNN architectures for Intrusion Detection in IoT networks, (ii) dynamic selection of the most suitable GNN configuration based on graph properties of the IoT network using the so-called "Adaptive Property-Aware Graph Neural Network (ADAP-GNN)". This method can deploy neighbor sampling safely in datasets with a large fraction of malicious nodes, as the risk of missing information about rare attacks is low. Also, attention mechanism proves advantageous in networks with high centrality entropy. Experimental results demonstrate the importance of property-aware model selection for developing robust GNN-based NIDS, highlighting the adaptability and effectiveness of the proposed approach compared to existing methods.

1. Introduction

Internet of Things (IoT) network security has become crucial in today's increasingly interconnected digital world. Identifying potential threats and attacks is a critical step to ensure the safety of internet connections and confidentiality of data [1]. One of the main tools to identify threats is the Network Intrusion Detection System (NIDS), which is designed to continuously monitor network traffic in order to detect any adversarial activity [2]. With the evolution of cyber threats and the complexity of modern networks,

[☆] The code is available at: <https://github.com/mtermos/ADAP-GNN>.

* Corresponding author at: CESI LINEACT, UR 7527, Strasbourg, France.

E-mail addresses: mtermos@cesi.fr (M. Termos), zghalmane@cesi.fr (Z. Ghalmane), abrahmia@cesi.fr (M. Brahmia), ahmad.fadlallah@ul.edu.lb (A. Fadlallah), ali.jaber@ul.edu.lb (A. Jaber), mzghal@cesi.fr (M. Zghal).

traditional rule-based NIDS methods have struggled to keep pace. This moved the focus of researchers to AI-based NIDS that use anomaly detection or train powerful classifiers.

Although Machine Learning (ML) and Deep Learning (DL) models have shown remarkable results [3], they are limited to examining each flow in isolation or as part of a short time sequence. Without exploring the relational pattern between devices and their connections in the networks, these models were blind against structured attacks that may look benign on a one-flow level [4]. To mitigate this challenge, graph machine learning was utilized to give models a broader view. A relatively new subfield of DL models that is tailored to deal with graphical data has been emerging called Graph Neural Networks (GNNs). GNN-based NIDS have shown state-of-the-art results compared to other DL models due to their ability to explore relational patterns in graphs [5]. Graph Neural Networks (GNNs) are designed to operate directly on graph-structured data, where nodes represent entities and edges show their relationships [6]. Using a message-passing approach, each node gathers information from its neighbors and updates its own representation with both its features and the context provided by its connections. This iterative process allows GNNs to learn high-dimensional node representations that reflect their place in the graph. In network traffic data, a graph can be built where devices are nodes and the flows between them are edges. Applying GNNs to such graphs can effectively capture complex interactions, making them a powerful tool for network intrusion detection systems [7]. For instance, GNN models are better at detecting PortScan attack, where attackers probe a network to identify open ports and vulnerable services. There are several architectures of GNN, and two widely adopted are GraphSAGE (Graph SAmple and AggreGAtE) [8] and Graph Attention Networks (GAT) [9]. GraphSAGE utilizes a neighbor sampling technique for scalable learning on large graphs and trains an aggregator function to ensure inductive node embedding. On the other hand, GAT leverages an attention mechanism to give weights to neighboring nodes.

Applying Graph Neural Networks (GNNs) to Network Intrusion Detection Systems (NIDS) has demonstrated superior performance compared to traditional deep learning models, particularly because IoT data can naturally be modeled as graphs. However, the current literature lacks a systematic investigation into which GNN architecture is most suitable for a given IoT network, based on its topological properties.

Our contribution is twofold. First, we address the mismatch between standard GNN architectures (which are typically node-centric) and the requirements of NIDS, where the focus is edge-centric. While E-GraphSAGE [10] has previously been proposed to adapt GraphSAGE for NIDS, our work goes further by introducing E-GAT, a novel edge-centric variant of the Graph Attention Network. This adaptation is non-trivial and required redesigning key architectural components, including reformulating edge attention mechanisms based on neighboring edge features and modifying the message-passing and aggregation processes.

Second, we propose a rule-based selection framework that determines the most suitable GNN architecture (either E-GAT, which leverages an attention mechanism, or E-GraphSAGE, which employs a neighborhood sampling mechanism) for any given IoT network dataset. This selection is based on two key proposed measures that reflect both the structural and security characteristics of the network: Degree Centrality Entropy and the Proportion of Attackers. These measures are computed at the node level and are chosen because they directly influence the relative performance of the two GNN mechanisms. Our framework performs this selection prior to training, ensuring ease of deployment, improved detection performance, and full interpretability. The rule-based nature of the approach guarantees transparency and avoids the added complexity of dynamic switching during training or inference. Unlike neural architecture search methods that require extensive training time and computational resources to explore large design spaces. Note that other GNN variants, such as GCN, GIN, and Graph Transformers, are not included in this study because their theoretical and practical incompatibility with edge-centric flow-based intrusion detection tasks.

Our contributions in this paper are:

- Extending the edge-variant GNN for NIDS by incorporating neighbor sampling in the E-GraphSAGE model.
- Proposing a new edge-variant E-GAT that incorporates attention mechanisms to enhance feature aggregation and adaptivity.
- Proposing ADAP-GNN, a rule-based model selection algorithm to choose the most suitable configurations of GNN for NIDS, according to the properties of the dataset and network.
- Conducting comprehensive experiments on nine NIDS datasets.
- Comparing real-time inference performance of different models.

We validate our framework through extensive experiments comparing different GNN configurations across nine diverse datasets. The results demonstrate that our rule-based selection method not only improves the effectiveness of network intrusion detection but also reduces computational costs when necessary.

The remainder of this paper is organized as follows: Section 2 reviews related works. Section 3 details the models used and introduces our methodology and the selection algorithm. We present the experimental evaluation in Section 4 and conclude the paper in Section 5.

2. Related work

This section reviews key studies on the use of graph-based approaches in Network Intrusion Detection Systems (NIDS), from hand-engineered complex networks measures to data-driven Graph Neural Networks (GNNs). After that, background on Graph Neural Networks and the process by which useful representations of the nodes are learned is provided. Finally, existing strategies for GNN architecture search are explored and their limitations in the context of intrusion detection are discussed.

2.1. NIDS strategies

Traditional signature-based NIDS rely on rule-based techniques to detect intrusions by matching known “signatures” of attacks. However, these methods are vulnerable to zero-day attacks — threats that do not match any pre-stored signature — and struggle to cope with the complexity of modern networks [11]. More recent work leverages machine learning and deep learning algorithms to learn complex patterns in network traffic data, with these approaches often showing improved performance classification accuracies [12].

Network traffic naturally forms graphs, where nodes represent entities such as devices and edges denote the connections between them [13]. The relational and topological patterns in these graphs are crucial for understanding the dynamics of the network [14,15]. Traditional flat machine learning and deep learning models overlook these patterns, missing important information [16]. Centrality measures are metrics used in complex network analysis to quantify the importance of individual nodes within a graph. Augmenting deep learning models with these centrality features has been shown to enhance the performance of flow classification in NIDS [17].

Graph Neural Networks (GNNs) were developed to directly process graph-structured data as artificial intelligence (AI) moved toward more powerful, deeper, and data-driven models [18]. Using a message-passing mechanism, they aggregate information from neighboring nodes, which allows them to capture both local and global patterns within the network. One of the most popular variants of GNN is a model called GraphSAGE (Graph SAmple and AggreGatE) that focuses on inductive and efficient learning [8]. GNNs were initially designed for general graph tasks like link prediction and node classification disregarding the edge features. However, in NIDS all information is focused on the edges in the context of network flows datasets. To overcome this challenge, new graph representations or changes to the architecture of GNN were proposed. For instance, Pujol-Perich et al. proposed modifying the graph representation of the data by creating an intermediate attributed node storing the flow statistics between the source and destination nodes. In this new representation of data, the GNN is capable of working normally as a node classifier [19]. In another work, Lo et al. modified the architecture of GraphSAGE to use edge features, introducing an edge variant called E-GraphSAGE. Their new model learns new node features by aggregating edge features and performing graph convolutions [10]. More refinements were applied to the E-GraphSAGE implementation, like adding the original edge features as a residual in the classification phase, which made the model more robust in classifying mixed-class flows between a pair of nodes [20]. Applying GNN to NIDS yielded superior performance in detecting attacks [5].

Despite these advances, selecting the best GNN architecture remains a significant challenge. Due to the diversity in network topologies and dataset properties, no single architecture consistently delivers optimal performance [5]. Additionally, different GNN architectures vary in their computational requirements [6], a critical factor in resource-limited environments. As a result, there is a clear need for guidelines to help choose the most suitable GNN configuration for a specific scenario. To address this gap, our work introduces a light-weight interpretable rule-based selection algorithm to choose the most appropriate GNN configuration based on the characteristics of the dataset and the graph of IoT devices and their connections. We focus on the E-GraphSAGE as our base model due to its state-of-the-art performance and explore how to improve its adaptability to different environments.

2.2. Graph neural networks

A Graph Neural Network (GNN) is a type of deep learning model designed to operate on graph-structured data. It was inspired by the way Convolutional Neural Networks (CNNs) recognize the spatial relations between pixels [21]. Pixels are located in a grid, and this notion can be generalized to more dynamic structures where nodes do not have a fixed number of neighbors. The goal was a GNN that focuses on the relational patterns between nodes and edges within a graph. In a typical graph-structured data, each node is represented as an embedding. After that, an iterative message-passing algorithm updates these embeddings based on information aggregated from neighboring nodes. By continuously refining node representations through this structure-aware process, GNNs capture complex topological relationships in an effective way, making them well-suited for tasks such as node classification and edge prediction. Two main architectures of GNN that are widely used are GraphSAGE and GAT, which will be presented in the following two sections.

2.2.1. GraphSAGE

GraphSAGE (Graph SAmple and AggreGatE) is a popular GNN variant that uses inductive node embedding and focuses on efficient and scalable training for large graphs [8]. Rather than using simple convolutions on the matrix factorization as in GCN, GraphSAGE trains a set of aggregator functions. This change leads to multiple advantages: aggregator functions can be applied to unseen nodes or new graphs, making GraphSAGE inductive rather than transductive. Also, samples of the neighboring nodes can be used to train the aggregator functions, making it more scalable on large graphs. Moreover, different aggregator functions can be used, making GraphSAGE more flexible.

The main step of GraphSAGE is to aggregate the embeddings of the sampled neighboring nodes N_v . At a layer k , all the embeddings neighboring nodes are aggregated to create one fixed-sized vector $\mathbf{h}_{N_v}^k$ as shown in Eq. (1).

$$\mathbf{h}_{N_v}^k = \text{AGG}^k(\{\mathbf{h}_u^{k-1}, \forall u \in N_v\}) \quad (1)$$

Here, AGG is a differentiable aggregation function that combines the embeddings of neighboring nodes into a fixed-size vector. Each layer k has its weights for the aggregation function AGG^k . $\mathbf{h}_u^{k-1}$ denotes the embedding vector of node u from the previous layer $k-1$. Node u is one of the nodes in the neighborhood N_v of node v .

The aggregated vector $\mathbf{h}_{N_v}^k$ is concatenated with the previous layer's embedding of node v . A fully connected layer transforms the concatenated vector into the updated embedding $\mathbf{h}_v^k$ for node v as shown in Eq. (2), where $\mathbf{W}^k$ is a trainable weight matrix, and σ is a non-linear activation function.

$$\mathbf{h}_v^k = \sigma \left(\mathbf{W}^k \cdot \text{CONCAT} \left(\mathbf{h}_v^{k-1}, \mathbf{h}_{N_v}^k \right) \right) \quad (2)$$

After K layers, the final embedding $\mathbf{h}_v^K$ of each node $v \in \mathcal{V}$ is denoted as $\mathbf{z}_v$.

2.2.2. Graph Attention Network (GAT)

Graph Attention Networks (GATs) extend the message-passing paradigm of GNNs by incorporating a learnable attention mechanism over neighbors, rather than applying uniform or fixed-weight aggregation [22]. The self-attention mechanism learns the relative importance of each neighbor's contribution to a node's representation. Calculating the attention is efficient, since it is parallelizable across neighboring nodes. Also, the model is applicable to inductive learning problems.

In GAT, attention coefficients $\alpha_{v,u}$ are computed to quantify the importance of each neighbor u in learning representation of node v . GAT uses self-attention on the neighboring nodes to compute raw attention coefficients. These coefficients are then normalized across all neighbors using a softmax function. This normalization ensures that the weights sum to one, forming a valid probability distribution that allows the model to focus more on the most relevant neighbors while reducing the influence of less significant ones.

$$\alpha_{v,u} = \frac{\exp \left(\text{LeakyReLU} \left(\bar{a}^T [\mathbf{W}\bar{h}_v \parallel \mathbf{W}\bar{h}_u] \right) \right)}{\sum_{k \in N_v} \exp \left(\text{LeakyReLU} \left(\bar{a}^T [\mathbf{W}\bar{h}_v \parallel \mathbf{W}\bar{h}_k] \right) \right)} \quad (3)$$

where $\bar{h}_v$ and $\bar{h}_u$ are the input feature vectors for nodes v and u , respectively; $\mathbf{W}$ is a weight matrix; $\bar{a}$ is the learnable attention weight vector; $\parallel$ denotes vector concatenation; N_v is the set of neighbors of node v .

In addition to computing attention coefficients, GAT aggregates neighbor features to update each node's representation. This is done using the following equation:

$$\bar{h}'_v = \sigma \left(\sum_{u \in N_v} \alpha_{v,u} \mathbf{W}\bar{h}_u \right) \quad (4)$$

where $\bar{h}'_v$ is the updated feature vector for node v , σ is a non-linear activation function. This equation completes the GAT layer by combining the learned attention coefficients with the transformed features of each neighboring node. Original GAT employs a multi-head attention mechanism. However, for simplicity and clarity, we focus only on the single-head mechanism in this work.

A GAT model is usually built of k layers, and at the last layer K , the final embedding $\mathbf{h}_v^K$ of each node $v \in \mathcal{V}$ is denoted as $\mathbf{z}_v$.

2.2.3. Adapting GNN models for network intrusion detection

Traditional GNN models, including GraphSAGE and GAT, were originally designed for tasks that use node features, like link prediction and node classification. However, in Network Intrusion Detection Systems (NIDS), the critical information is embedded in the edges representing network flows, and all datasets in this field are composed of flows between two IP addresses. This fundamental difference necessitates an edge-centric approach that uses the edge features instead of the node features [23]. To make GNN edge-centric, E-GraphSAGE was proposed by Lo et al. [10]. In this variation, the model is trained to learn an aggregator function that collects and integrates information from edges to generate new node embeddings. Using this aggregation method, each node's embeddings are influenced by its surrounding context, enabling the GNN to capture and explore the complex graphical patterns in the IoT network traffic. This aggregation function is the core functionality of the GNN, but the implementation was limited to one type of aggregation. In previous works, E-GraphSAGE is limited to uniform weighting and to using the full neighborhood. Considering these limitations, we propose edge-centric GNN models that can use the neighbor sampling technique, which reduces inference time, and the attention mechanism, which dynamically weighs the importance of node contributions while aggregating. This property-aware implementation and additional choices in configuring edge-centric GNN models form the basis for our proposed rule-based selection algorithm, which tailors GNN configurations to graph properties for optimal performance in NIDS.

2.3. GNN configuration strategies

Graph Neural Networks (GNNs) have shown great performance across a variety of graph-structured tasks, but their effectiveness is largely dependent on the architectural choices and hyperparameters. In domains like Network Intrusion Detection Systems (NIDS), where the input graphs vary significantly in topology, density, and flow patterns, selecting the right GNN configuration becomes particularly critical. A poor configuration may lead to underfitting in some environments or overfitting in others. To make this decision, a number of methods have been proposed to search for the most suitable GNN architectures for a given task or dataset. In deep learning, Neural Architecture Search (NAS) is used to discover effective deep architectures. However, NAS cannot be applied to GNN directly. For that reason, multiple works have proposed adaptations of NAS in GNN. One of these methods is GraphNAS [24], which utilizes reinforcement learning to maximize the expected accuracy of the generated architectures. GraphNAS formulates the search process as a reinforcement learning problem, where a recurrent neural network controller generates candidate GNN architectures by selecting operations such as the number of layers, aggregation functions, activation functions, and

Table 1

Overview of network intrusion detection datasets used in the evaluation experiments, including: record counts, attack types, percentage of records that are not benign, and dataset descriptions.

Dataset	Size	Attacks	Percentage of attacks	Description	Year
CIC-ToN-IoT [26]	5,350,583	DoS, DDoS, MITM, Scanning, Injection, Password, Backdoor, Ransomware, XSS	53.01%	CICFlowMeter representation of the TON-IoT dataset focuses on evaluating cybersecurity applications in the Internet of Things (IoT).	2020
CIC-ToN-IoT-Snapshot [27]	4,915,090	Scanning, Injection, Password, Backdoor, Ransomware, XSS	57.24%	A modified CIC-ToN-IoT dataset, extracted from a specific time window, preserves original network relationships and maintains realistic node attributes.	2020
CIC-IDS-2017 [28]	2,814,878	DoS, DDoS, Bot, PortScan, Infiltration, Web attacks, Patator, Heartbleed	19.5%	A modern, realistic network simulation for testing intrusion detection and prevention systems using benign traffic.	2017
NF-UQ-NIDS [29]	10,729,039	Exploits, DoS, DDoS, Reconnaissance, Generic, Scanning, Injection, Password, Backdoor, XSS, Ransomware	21.33%	A compilation of multiple network flow datasets creates a comprehensive benchmark for network intrusion detection, featuring diverse scenarios.	2021
NF-UQ-NIDS-Unified-Community [27]	2,203,216	Exploits, DoS, DDoS, Reconnaissance, Generic, Worms	29.89%	A modified NF-UQ-NIDS dataset, created through community mixing that unifies its structure while preserving the original balance of traffic types.	2021
NF-CSE-CIC-IDS2018 [29]	7,314,661	Infiltration, Bot, Brute force, DDOS, DoS	8.64%	A NetFlow-based dataset offering a comprehensive view of network behavior, capturing diverse flow patterns to support advanced analysis and detection.	2018
CCD-INID-Centrality-Focused [27]	64,199	MITM, ARP DoS, UDP DoS, Hydra brute, Slowloris	47.95%	A refined IoT network dataset utilizing centrality analysis to emphasize key nodes for enhanced security in the original CCD-INID dataset [30].	2021
Edge-IIoTset [31]	2,031,921	MITM, Ransomware, SQL injection, DDoS, Password, Port scanning, Backdoor, XSS	29.35%	A diverse industrial IoT dataset selected for ML and DL experiments provides valuable traffic records representing benign and attack activities.	2022
X-IIoTID [32]	631,814	Reconnaissance, RDoS, Exploitation, Tampering, C&C, Ransomware	48.59%	An extensive, connectivity-agnostic dataset addresses Industrial IoT security challenges by recording multiview features of network traffic, resources, logs, and warnings.	2021

hidden dimensions. Each candidate model is trained and evaluated, and the feedback is used to update the controller's policy. This method was primarily evaluated on standard node classification benchmarks like Cora and Citeseer, and it assumes the presence of node features and a static graph structure. In another work, Auto-GNN [25] was proposed as a general framework for automatic GNN design. It combines architecture search and hyperparameter optimization by using reinforcement learning or gradient-based controllers to explore the space of possible GNN models. The goal is to find the best-performing architecture for a given dataset. Auto-GNN has been shown to outperform manually designed models in several tasks such as node classification and graph classification. However, its focus remains on node-centric tasks using node features and static graphs like GraphNAS. To the best of our knowledge, its application to flow-based NIDS has not been explored.

Despite their effectiveness, both Auto-GNN and GraphNAS present several limitations when applied to flow-based intrusion detection in IoT networks. First, they are designed for node-level tasks and are not natively adapted to edge-level classification, where the target is to label flows (edges) rather than nodes. Second, they rely on node features for representation learning, which are often missing or synthetic in NIDS datasets. Third, both methods introduce high computational overhead, requiring extensive search-time training to evaluate multiple candidate architectures. This makes them impractical for deployment in real-time or resource-constrained environments. To address these limitations, we propose ADAP-GNN, a lightweight, interpretable, and domain-specific alternative. Instead of exhaustively searching a large design space, ADAP-GNN uses two simple yet powerful graph-level metrics “proportion of attackers” and “degree centrality entropy” to dynamically select between two carefully adapted edge-centric architectures: E-GraphSAGE and E-GAT. These models are designed specifically for flow-based intrusion detection, and our framework avoids the overhead of neural architecture search while offering strong performance and high interpretability.

3. Proposed method

In this work, we propose an implementation of edge-centric GNN variants that can utilize the neighbor sampling technique and the attention mechanism. Also, we introduce Adaptive Property-Aware Graph Neural Network (ADAP-GNN), a framework to select the most suitable GNN configuration according to the graph properties of the dataset. In this section, the edge-centric variants of GraphSAGE and GAT, the so-called E-GraphSAGE and E-GAT models, are explained. Then, the graph properties used in the selection algorithm are described. After that, the general scheme of ADAP-GNN is presented, and the rule-based selection algorithm is detailed. Finally, toy examples are provided to illustrate the intuition behind the selection algorithm.

3.1. Graph neural network model for NIDS

To adapt GraphSAGE for network flow classification, the E-GraphSAGE model was introduced as an edge-centric variant tailored for NIDS, where only edge features are given [10]. Since network flow datasets typically lack inherent node features, node representations are initially set to a constant vector. Over K layers, each node updates its embedding by aggregating the edge features from its neighbors using a learnable aggregation function. Once the node representations have been refined, edge embeddings are computed by combining the source and destination node embeddings with the corresponding edge features, followed by classification using a Fully Connected (FC) layer. In later works, it was demonstrated that including the original edge features in the classification phase leads to improved results [20]. This enhancement is particularly important because multiple network flows — potentially belonging to different classes — can exist between the same pair of nodes. Without the original edge features, the model will assign a single class to all flows between a given source and destination. This makes the model capabilities limited in cases where mixed flow classes occur. By integrating the original edge information, the E-GraphSAGE model can personalize each edge representation, resulting in more accurate classification.

Furthermore, while the original implementation considered all the neighboring nodes in the aggregation step, we have added the neighbor sampling strategy into our implementation. This modification enables more efficient and targeted aggregation, especially in large-scale network graphs. Our aim is to study when is using neighbor sampling is the best choice, according to the dataset and graph properties. The overall procedure of E-GraphSAGE is summarized in Algorithm 1, outputting a final edge embedding that captures both the network's structural information and the detailed flow-based characteristics. For more clarity, Fig. 1 illustrates the aggregation of edge features to update the representation of the nodes in the E-GraphSAGE layer and how the new features are used to classify edges in the classification phase.

Algorithm 1 E-GraphSAGE Model

Require: Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$,

Edge features $\mathbf{e}_{uv}$ for each $uv \in \mathcal{E}$,

Depth K of the GNN model,

Non-linear activation function σ ,

Aggregation function AGG

Ensure: New edge embeddings $\mathbf{z}_{uv}$ for each $uv \in \mathcal{E}$

```

1: Initialize node features  $\mathbf{h}_v^0$  as a vector of ones  $\forall v \in \mathcal{V}$ 
2: for  $k = 1$  to  $K$  do
3:   for each node  $v \in \mathcal{V}$  do
4:      $\mathbf{h}_{N(v)}^k = \text{AGG}^k(\{\mathbf{e}_{uv}^k, \forall u \in N(v)\})$ 
5:      $\mathbf{h}_v^k = \sigma(\mathbf{W}^k \cdot \text{CONCAT}(\mathbf{h}_v^{k-1}, \mathbf{h}_{N(v)}^k))$ 
6:   end for
7: end for
8: Final node embeddings:  $\mathbf{z}_v = \mathbf{h}_v^K \forall v \in \mathcal{V}$ 
9: for each edge  $uv \in \mathcal{E}$  do
10:   $\mathbf{z}_{uv} = \text{AGG}_{edge}(\mathbf{z}_u, \mathbf{z}_v, \mathbf{e}_{uv})$ 
11: end for
12: return  $\mathbf{z}_{uv}$  for each  $uv \in \mathcal{E}$ 

```

Building upon the edge-centric framework of E-GraphSAGE, the E-GAT model is proposed, which integrates a single-head attention mechanism to further refine the aggregation process. E-GAT leverages attention to weight the contributions of neighboring nodes to create better node embeddings that will be used in the classification of the flows. As with E-GraphSAGE, the model initializes node features as constant vectors due to the absence of inherent node attributes. Over K layers, for each edge, the source and destination node features are concatenated to form an intermediate representation. This representation is then transformed by a learnable attention weight matrix and an activation function to produce raw attention scores. After that, a Softmax function is used to normalize these scores across all neighbors. The resulting coefficients are used to weight the edge embeddings during the aggregation. This allows the model to focus on connections from nodes that would be more informative. The message passing procedure in the E-GAT model is detailed in Algorithm 2. After iteratively updating the node embeddings through attention-weighted aggregation, the final embeddings are used to classify each edge.

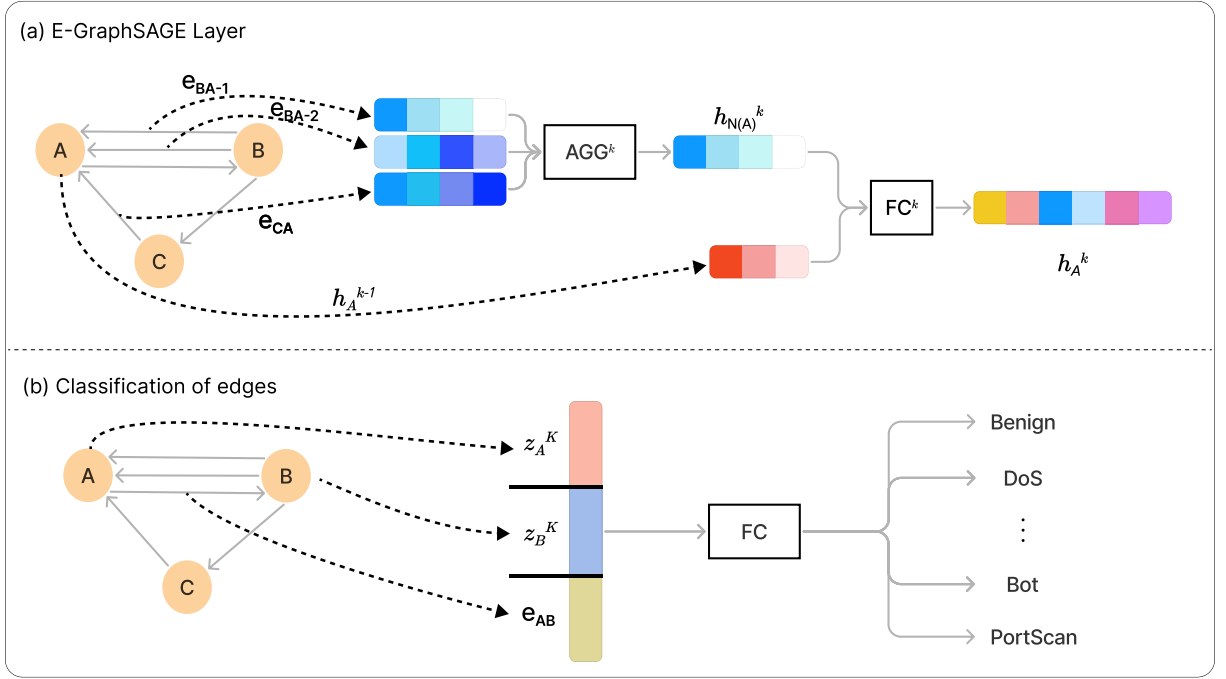


Fig. 1. Illustration of the E-GraphSAGE layer and the classification phase after it. At a layer k , a node A updates its representation by aggregating edge features, such as e_{BA1} and e_{BA2} from its neighbor B . After K iterations, the final embeddings z_A and z_B are obtained. In the classification phase, original edge features e_{AB} is used alongside the new obtained node embeddings to classify the edge as a benign activity or an attack.

Algorithm 2 E-GAT Model

Require: Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$,

Depth K of the GNN model,

Edge features e_{uv} for each $(u, v) \in \mathcal{E}$,

Non-linear activation function σ

Ensure: New edge embeddings z_{uv} for each $uv \in \mathcal{E}$

1: Initialize node features h_v^0 as a vector of ones $\forall v \in \mathcal{V}$

2: **for** $k = 1$ **to** K **do**

3: **for** each edge $uv \in \mathcal{E}$ **do**

4: $e_{uv}^{(k)} = \text{CONCAT}(h_u^{(k-1)}, h_v^{(k-1)})$

5: $e_{uv}^{(k)} = \text{LeakyReLU}(\mathbf{W}_{\text{att}}^{(k)} \cdot z_{uv}^{(k)})$

6: **end for**

7: **for** each node $v \in \mathcal{V}$ **do**

8: $\alpha_{uv}^{(k)} = \frac{\exp(e_{uv}^{(k)})}{\sum_{u \in N(v)} \exp(e_{uv}^{(k)})} \quad \forall u \in N(v)$

9: $h_{N(v)}^k = \sum_{u \in N(v)} \alpha_{uv}^{(k)} z_{uv}^{(k)}$

10: $h_v^k = \sigma(\mathbf{W}^k \cdot \text{CONCAT}(h_v^{k-1}, h_{N(v)}^k))$

11: **end for**

12: **end for**

13: Final node embeddings: $z_v = h_v^K \quad \forall v \in \mathcal{V}$

14: **for** each edge $uv \in \mathcal{E}$ **do**

15: $z_{uv} = \mathbf{W}^{\text{edge}} \cdot \text{CONCAT}(z_u, z_v, e_{uv})$

16: **end for**

17: **return** z_{uv} for each $uv \in \mathcal{E}$

Other GNN variants such as GCN, GIN, and Graph Transformers were not considered in our experiments. Classic GCNs are transductive and aggregate only normalized neighbor messages without concatenating node features, making them unsuitable for edge-centric tasks without extensive modifications. Moreover, an edge-centric GCN reduces to a special case of E-GraphSAGE with

full neighborhood aggregation. GIN models, although powerful for node or graph classification, use injective sum aggregation without concatenating original features, which again fails to accommodate edge-level flow information. Similar to GCN, adapting GIN to run in edge-centric NIDS would need architectural modifications that would make it the same as E-GraphSAGE. Finally, Graph Transformers employ global self-attention and positional encodings, leading to quadratic complexity in the number of nodes. Adapting them to sparse, edge-centric intrusion detection graphs would require non-trivial redesign. For these theoretical and practical reasons, we focus on E-GraphSAGE and E-GAT, which are better aligned with our graph construction of the flow-based NIDS.

3.2. Graph and dataset properties

Networks in real life have different properties; two networks would be different in terms of the number of devices, numbers and types of attacks expected, and the graph structure. Also, the number of connections and importance of nodes in the graphs would be different. Moreover, different environments would have different computation resources, such as IoT network and a network of devices in a large enterprise with powerful servers. With different choices of model architecture, it is important to choose the most suitable architecture for a situation, as some choices, such as attention mechanism used by E-GAT, having a higher computational cost. In this study, metrics utilized can describe the properties of the dataset itself and the graph constructed by the connections between IoT devices. These metrics are presented in the following subsections:

3.2.1. Proportion of attackers

Proportion of attackers is a very critical metric when considering graph related information in NIDS. It is the ratio of nodes that have initiated an attack to the total number of nodes in the network, resulting a value between zero and one. A low ratio indicates that only a few nodes are responsible for attacks, while a high ratio means that a high number of attackers relatively to the total number of nodes. This metric is critical in GNN where information from neighboring nodes is aggregated, and attackers may be any of them. Neighbor sampling is a technique used in E-GraphSAGE to reduce computational costs and improve generalizability [33], but there is a risk of completely excluding attacker nodes. If these nodes are underrepresented during training, the model will fail to learn crucial patterns of adversarial behavior. Therefore, we hypothesize that the proportion of attackers significantly influences the effectiveness of neighbor sampling technique used in E-GraphSAGE. The proportion of attackers, denoted as p , is defined by the equation:

$$p = \frac{N_{\text{attackers}}}{N_{\text{total}}} \quad (5)$$

where $N_{\text{attackers}}$ is the number of nodes that have initiated an attack and N_{total} is the total number of nodes in the graph.

3.2.2. Centrality entropy

In graph theory and complex networks, centralities are measures that quantify the importance of a node in the graph. They provide a ranking for all nodes in the graph [34]. Since “importance” can hold different meanings, there exist two types of centralities: local and global. Local centralities focuses on the relation of a node in its close neighborhood, such as the number of its connections. On the other hand, global centralities focus on the place of the node in the whole graph and how it connects different parts of the graph together. For instance, a node may have few connections, but these connections are vital for passing information to different parts of the graph.

While the centralities are node-level measures, to use them in deducing the graph properties, the “entropy” concept is utilized to study their distribution. In information theory, entropy captures the variability or uncertainty in the distribution of a score, considering the distribution of probabilities across all potential states [35]. To ensure that value is normalized between 0 and 1, the entropy is then divided by the maximum possible entropy in the graph. To calculate the entropy E of a centrality, the following equation is used:

$$E = \frac{-\sum_c^C p_c \log(p_c)}{\log(|\mathcal{V}|)} \quad (6)$$

where c denotes an element of the set C , the set of all unique centrality values available in this graph. p_c is the count of occurrences of a certain value of centrality measure for different nodes in the graph. A maximum entropy possible is achieved if each node has a unique centrality value, thus the total number of nodes in the graph $|\mathcal{V}|$ is used for normalization.

Calculating the centrality entropy gives us an overview of the types of connections and distribution of influence in the graph. If a graph has a low entropy, then the centrality values have a very close range, except for some outliers. However, if a graph has a high centrality entropy, then the values of the centrality are spread out; there are nodes with varying importance, but no clear dominance for a few nodes.

Our objective is to use this measure to decide whether or not to use the attention mechanism in the aggregation phase of the edge-centric GNN model. In networks with low centrality entropy, the basic message-passing method of a Graph Neural Network is generally enough to gather useful information from neighboring nodes. Using an attention mechanism in these cases might cause the model to overemphasize a few outlier nodes, leading to overfitting. In other words, the model becomes overly specialized to the specific characteristics of these nodes, reducing its ability to generalize to new data. However, in networks with high centrality entropy — where there is a wide variety of node importance — the attention mechanism becomes essential. High entropy indicates

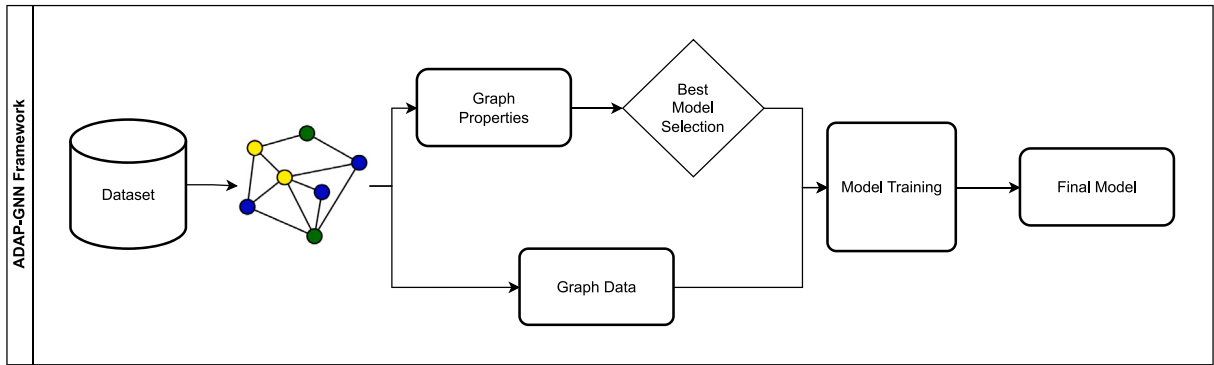


Fig. 2. The general scheme of the proposed ADAP-GNN. The rule-based GNN selection framework starts by constructing the graph from the network flows. Then extracting key properties from the dataset and graph. These results are used to select the best model for the dataset, which will be trained on the graph data to classify flows and detect intrusions.

uncertainty in node significance, so a method that weighs each node differently helps the model capture the diverse contributions of its neighbors. In such cases, a more advanced aggregation technique is needed to effectively combine the information.

While entropy can be applied using various centrality measures, degree centrality is chosen in this paper because it requires minimal computational time [36], making it an efficient metric to apply in network intrusion detection systems. The degree of a node is the number of connections it has with other nodes in the graph. In our study, the type of the graph is MultiDiGraph, so the node's degree is defined as the sum of its in-degree and out-degree, with parallel edges counted as separate connections. Degree Centrality is a local centrality measure that quantifies the importance of a node based on its degree, normalized by the total number of nodes [14]. The Degree centrality C_D of a node i can be calculated as:

$$C_D(i) = \frac{d_i}{n-1} \quad (7)$$

where d_i denotes the degree of node i , and n represents the total count of nodes in the network.

To apply entropy of Degree centrality, a set of all unique values of Degree centrality C_D must be created. Then the probability of each unique value to exist in the dataset is calculated. Finally, Eq. (6) is applied on Degree centrality:

$$E_D = \frac{-\sum_c^{C_D} p_c \log(p_c)}{\log(|\mathcal{V}|)} \quad (8)$$

where $C_D = \{C_D(i) : i \in \mathcal{V}\}$ is the set of all unique values of Degree centrality. Each unique value is denoted by c , and p_c is the probability of that value to exist in the graph. $|\mathcal{V}|$ is the total number of nodes in the graph, the highest possible entropy for a graph.

Before settling on the proportion of attackers and degree centrality entropy, we evaluated over twenty graph metrics and dataset-level measures, including Average Degree, Graph Density, Betweenness Centrality Entropy, and Distribution Skewness of other centrality measures. Each candidate metric was assessed using a depth-1 decision tree: if the metric produced a pure split between configurations with neighbor sampling or attention mechanism and the baseline configuration with full neighborhood and uniform weights, it was kept; otherwise, it was discarded. Only the Proportion of Attackers and the Degree Centrality Entropy consistently yielded clear, interpretable splits across all datasets.

3.3. ADAP-GNN: General scheme

To outline the step-by-step process of our rule-based GNN selection framework for NIDS, a flow chart is presented in Fig. 2. Our approach starts by constructing the graph where nodes represent the IoT devices and edges represent the network flows between them. All flow-based features found in the dataset are considered as the edge features in the graph. Then, the properties of the dataset and graph are analyzed, including some graph properties such as the proportion of attacks and the entropy of degree centrality (Entropy can be applied to various centrality measures, but degree was used due to its minimal computational time). Based on these properties, the framework selects the optimal GNN architecture. Finally, the selected model is trained on the graph-structured dataset, improving detection accuracy and resource allocations. Additionally, systematic instructions are given as pseudo-code in Algorithm 3.

Algorithm 3 ADAP-GNN: Adaptive Property-Aware Graph Neural Network, to select GNN configurations using graph properties for NIDS

Require: Dataset D with labeled flows

Ensure: Trained edge-centric GNN model M

```

1:  $D \leftarrow \text{Preprocess}(D)$ 
2: Remove duplicates
3: Handle missing/infinite values
4: Normalize and encode features
5:  $G \leftarrow \text{ConstructGraph}(D)$  {Nodes = IP addresses, edges = flows}
6: Compute  $n_{\text{attackers}} \leftarrow$  number of nodes that initiate attacks
7: Compute  $n_{\text{total}} \leftarrow$  total number of nodes in  $G$ 
8:  $p \leftarrow n_{\text{attackers}}/n_{\text{total}}$  {Proportion of attackers}
9: Compute degree centrality  $C_D(i)$  for each  $i \in V(G)$ 
10: Compute degree entropy  $E_D$  using Eq. (6)
11: {Thresholds determined via logistic regression:  $p^* = 0.022$ ,  $E_D^* = 0.344$ }
12: if  $p > p^*$  then
13:   Use neighbor sampling in E-GraphSAGE/E-GAT
14: else
15:   Aggregate from the full neighbourhood
16: end if
17: if  $E_D > E_D^*$  then
18:   Employ attention mechanism (E-GAT)
19: else
20:   Use uniform weighting (E-GraphSAGE)
21: end if
22: Train the selected model  $M$  on  $G$ 
23: return  $M$ 

```

3.4. ADAP-GNN: Selection algorithm

Since no single graph neural network architecture had the optimal results on all datasets, we propose a light-weight interpretable rule-based selection algorithm. Unlike neural architecture search methods that require extensive training time and computational resources to explore large design spaces, we rely on the properties of the graph to have insights on what is the most suitable model for such a dataset before the training starts. Our rule-based selection algorithm ADAP-GNN uses both the proportion of attackers and degree centrality entropy to choose the most suitable GNN configuration, as illustrated in Fig. 3. First of all, we will investigate the use of the neighborhood sampling technique in E-GraphSAGE. This technique improves generalizability and reduces computational costs. If the network has a high proportion of attackers among its nodes, it is better to use neighbor sampling in the GNN model. However, if the proportion of attackers is low, then the full neighbor should be considered. Secondly, the attention mechanism used in E-GAT is investigated. This mechanism allows the model to weight the contributions of neighboring nodes, allowing the model to detect complex patterns and to focus on beneficial information. This attention mechanism should not be used if most of the nodes have the same importance and number of connections in the network. On the other hand, it should be used if the nodes have different numbers of connections, where additional information should guide the model in aggregating information from neighboring nodes.

As the number of devices that initiated an adversarial activity in a network increases, the neighbor sampling used in E-GraphSAGE model is safer to use, since the stochastic sampling is less likely to exclude all the attackers. However, if the number of attackers is low relative to the total number of devices, neighbor sampling would not be safe. Especially in imbalanced datasets and environments where each attack incident should be studied in order to create the classifiers with optimal performance at distinguishing attacks from normal behavior on the internet. We did not rely only on the imbalance of the dataset, since some balanced datasets with more attack records than benign have these attacks issued from a few nodes. And as the model uses neighbor sampling, all the attack records concentrated in the connections from these nodes have a high chance to be ignored altogether. As the neighbor sampling uses fewer computational resources, we should use it in the GNN models when it is safe and the computational resources are limited.

The attention mechanism used in E-GAT model helps improve the way a Graph Neural Network gathers information by giving more importance to neighbors that provide useful data. However, if used in the wrong situation, it may focus too much on a few key nodes during training, overfitting on the patterns where dominant nodes exist. In these cases, the model performs well on the training data but not as well on new, unseen data. To address this issue and make a guideline for when to use this mechanism in the model, we use the centrality measures and compute their entropy to assess the variability in the importance of nodes in the graph. High centrality entropy indicates that nodes have different values for centrality; this suggests that the attention mechanism can effectively differentiate between different nodes. Conversely, low centrality entropy indicates that nodes have similar importance, except for a few outliers. This suggests that the attention mechanism may overfit on the outliers. For that reason, a simpler aggregation method may be more efficient and less prone to overfitting.

By combining these two metrics, our algorithm selects the optimal configuration for the GNN model. If the proportion of attackers is high, then neighbor sampling is employed to reduce the computational overhead without leaving out critical information about

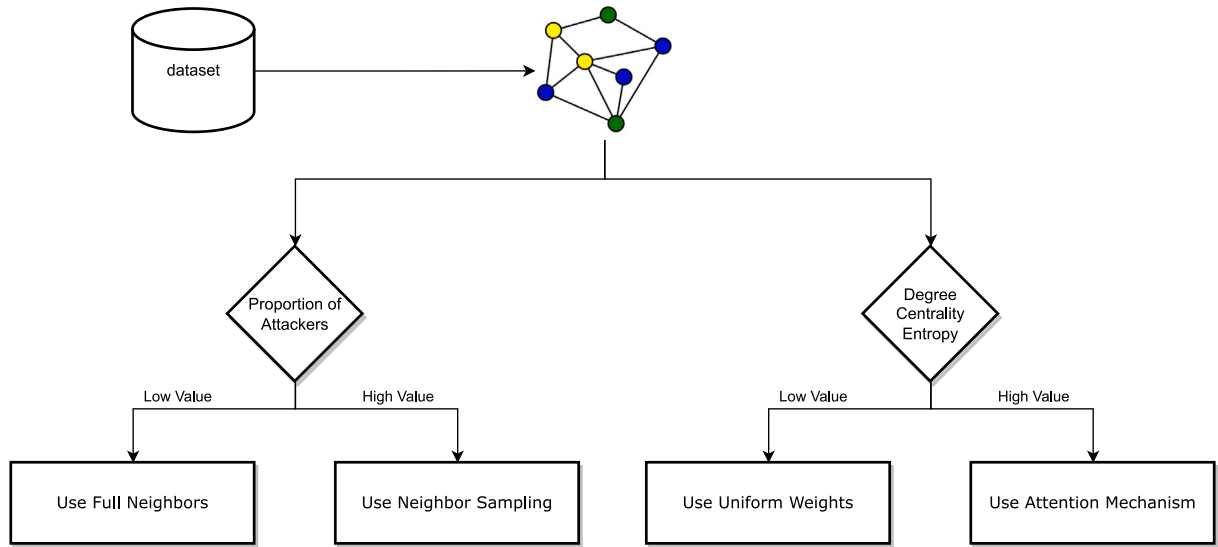


Fig. 3. ADAP-GNN selection algorithm of the most suitable GNN architecture according to the dataset and graph properties. The “Proportion of Attackers” property is used to configure neighbor sampling, while the “Degree Centrality Entropy” property is used to select the weighting mechanism in the aggregation process.

attacks. If the network has a high entropy in degree centrality, then a single-head attention mechanism is employed to weight the contributions of heterogeneous nodes. This adaptive strategy of ADAP-GNN ensures that the model is efficient as well as robust across diverse network scenarios.

3.5. Toy example

To illustrate the relation between the graph properties and the configuration of GNN models, toy examples are provided. In addition to the original GNN configuration, neighbor sampling and attention mechanism can be used. In the first toy example shown in Fig. 4, the effect of the proportion of attackers on the neighbor sampling technique is explained. In the second toy example displayed in Fig. 5, the relation between degree centrality entropy and the attention mechanism is illustrated. In these figures, small graphs were used to give a clear illustration of how different graph properties affect the aggregation process at a node v with its neighbors $u_1, u_2, \dots, etc$ using different GNN configurations. To represent nodes, we used circles, where red color means that the node has made an attack, while yellow means that all the connections issued from this node are benign activities. In case the nodes have different sizes, then the bigger the size is, the higher the importance of the node in terms of the number of connections (i.e., degree centrality). In the figures, arrows are used to represent the edges connecting these nodes. The red color represents attacks, while the yellow color represents normal activity.

3.5.1. Neighbor sampling toy example

The effect of neighbor sampling on the performance of the model is connected to the proportion of attackers in the network. In the toy example represented in Fig. 4, the impact of the proportion of attackers on the neighbor sampling technique is illustrated. In Fig. 4(a), the graph contains only two attackers, which is a low number compared to the thirteen nodes in the graph. This yields a proportion of attackers to be 0.15, which is low for such a small graph. Applying a stochastic neighbor sampling that randomly selects a few neighboring nodes may lead to excluding critical information about attackers and adversarial activities, as shown in the figure. On the other hand, the aggregation process in a graph with a higher number of attackers will be able to include information about attackers even after the neighbor sampling. The graph illustrated in Fig. 4(b) contains thirteen nodes where five are attackers; this means that the proportion of attackers is 0.38. As shown in the figure, the neighbor sampling did not exclude all attackers in the neighborhood of the node v .

3.5.2. Attention mechanism toy example

In a GNN model, the aggregation step is how the model uses the structure of the graph to update each node’s information. Adding the attention mechanism will increase the awareness of the model to the relevance of each neighboring node in learning new representations. A very powerful addition compared to a blind uniform weighting aggregation that considers each neighboring node has similar importance. However, this extra feature increases the model’s complexity and can lead to overfitting, where a model becomes specialized in specific cases and patterns seen in the training data and fails to generalize to new data. To decide when to use the attention mechanism, the degree centrality entropy measure is used. To illustrate the impact of this measure of

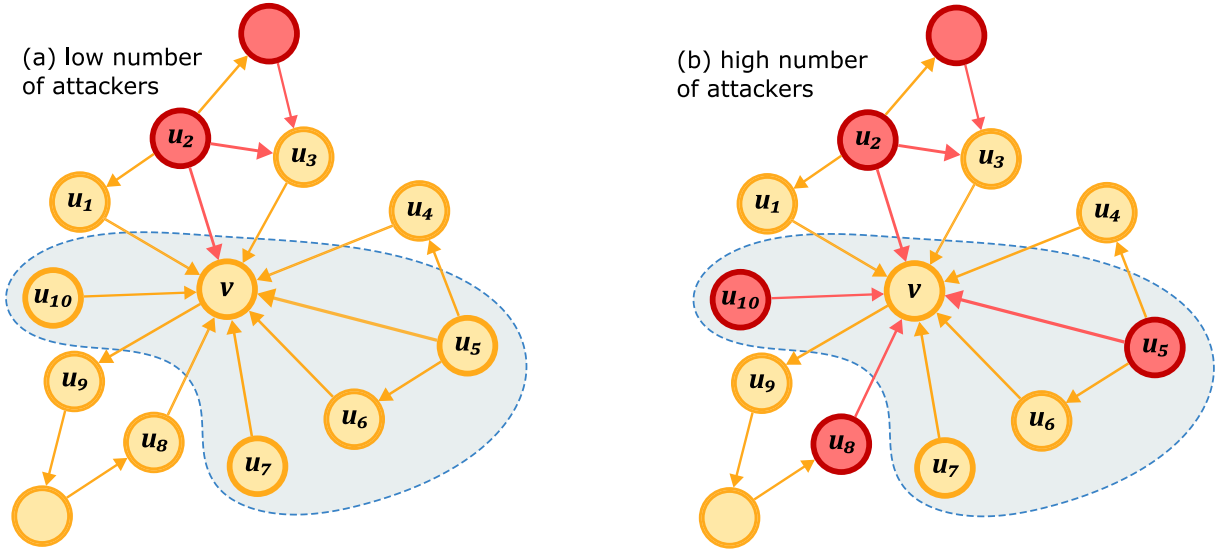


Fig. 4. Toy example illustrating the effect of the proportion of attackers on neighbor sampling. Red nodes represent attackers, and red arrows represent attack incidents, while yellow nodes are normal devices, and yellow arrows are benign activities. The blue area represents the nodes selected using neighbor sampling before aggregating features. In graph (a), all attacks and attackers are missed due to neighbor sampling. In graph (b), some attacks are included in the aggregation and then in classification, even while applying neighbor sampling.

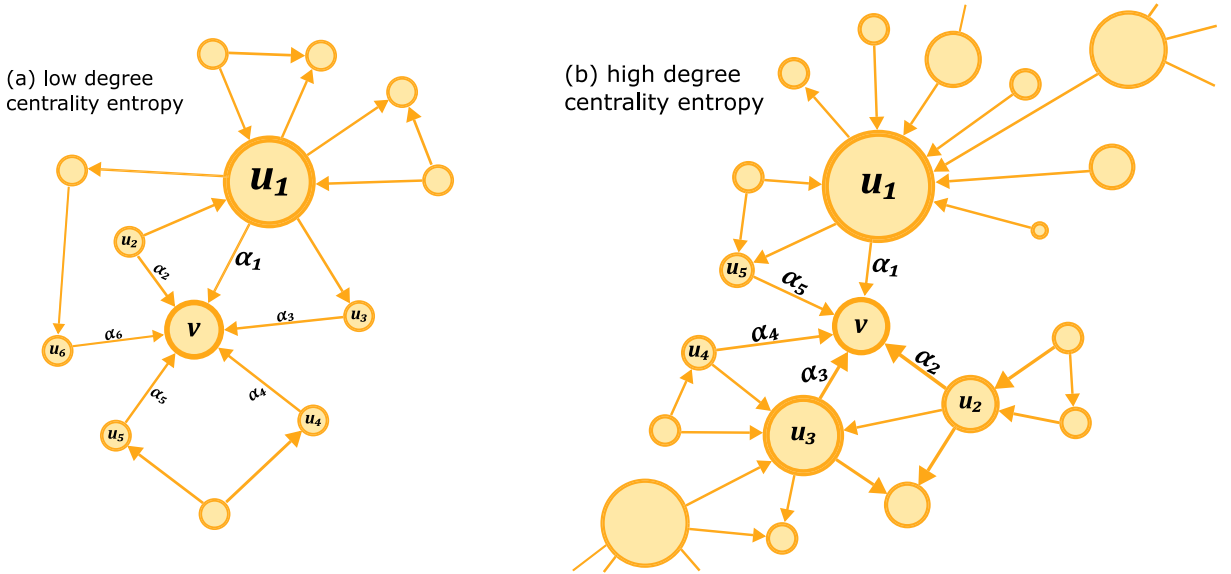


Fig. 5. Toy example displaying two cases of graphs with two different structures and degree centrality entropy. Bigger size indicates a bigger degree of centrality. In the graph (a), all nodes except v and u_1 have a degree of value 2. A model applying an attention mechanism will learn patterns that involve a dominant node such as u_1 , which will make the model specialized to specific use cases. On the other hand, a graph with high degree centrality entropy — most nodes have similar numbers of connections — is presented in Fig. 5(b). In this graph, nodes have different values of degree centrality, and no single node dominates. In such a case, the attention mechanism becomes more useful because it can effectively weigh the contributions of diverse nodes without overemphasizing a few dominant ones.

the attention mechanism, two graphs with different entropy values are illustrated in Fig. 5. A graph with a low degree centrality entropy — most nodes have similar numbers of connections — is presented in Fig. 5(a). In this graph, all nodes other than v and u_1 have a degree of value 2. A model applying an attention mechanism will learn patterns that involve a dominant node such as u_1 , which will make the model specialized to specific use cases. On the other hand, a graph with high degree centrality entropy — most nodes have similar numbers of connections — is presented in Fig. 5(b). In this graph, nodes have different values of degree centrality, and no single node dominates. In such a case, the attention mechanism becomes more useful because it can effectively weigh the contributions of diverse nodes without overemphasizing a few dominant ones.

Table 2

Graph properties and ADAP-GNN configuration decisions for datasets used in the experiments. For each dataset, the table shows the proportion of attackers and degree centrality entropy, along with the selected configuration for neighbor sampling and attention mechanisms.

Dataset	Proportion of attackers	Neighbor sampling	Degree centrality entropy	Attention mechanism
CIC-IDS-2017	0.00052	×	0.4405	✓
CIC-ToN-IoT	0.000077	×	0.174882	×
CIC-ToN-IoT-Snapshot	0.00012	×	0.176	×
Edge-IIoTset	0.776	✓	0.022	×
NF-UQ-NIDS	0.0079	×	0.2402	×
NF-UQ-NIDS-Unified-Community	0.073620	✓	0.688	✓
X-IIoTID	0.0803	✓	0.829	✓
NF-CSE-CIC-IDS2018	0.007862	×	0.239	×
CCD-INID-Centrality-Focused	0.475728	✓	0.815	✓

3.6. Computational complexity

It is important to understand the computational overhead of each GNN variant. For a K -layer model with $|V|$ nodes, $|E|$ edges and feature dimension d , E-GraphSAGE with full neighborhood aggregation has time complexity $\mathcal{O}(K|E|d)$ during training and inference. When neighbor sampling is employed with S sampled neighbors per node, the complexity reduces to $\mathcal{O}(K|V|Sd)$, trading accuracy for efficiency. Our novel E-GAT incurs a higher cost due to attention: computing attention coefficients for all edges costs $\mathcal{O}(|E|d)$ and aggregating weighted messages costs $\mathcal{O}(|V|d^2)$ per layer. Thus, E-GAT has a total complexity of $\mathcal{O}(K(|E|d + |V|d^2))$. Finally, ADAP-GNN's selection overhead is negligible, as it involves computing graph-level metrics ($\mathcal{O}(|V| + |E|)$) and two simple comparisons before training.

4. Experimental evaluation

In this section, the experimental evaluations of E-GraphSAGE and E-GAT in NIDS are represented. Also, the proposed ADAP-GNN is tested on nine datasets, each varying in size, topology, and format. First, the datasets and their properties are described. After that, the experimental setup is specified. Finally, the results are presented, analyzed, and compared, with insights and discussion offered.

4.1. Datasets

In our experiments, nine datasets with different characteristics and topological properties are used to evaluate the proposed ADAP-GNN. To give a general overview of each dataset, Table 1 presents the size of the dataset representing the number of records available in the CSV files. Out of the total number of records, the percentage of them classified as attacks is given alongside the main attack classes and types. Finally, a short description is provided for each dataset.

Although most of the datasets used in our evaluation are publicly available, several of them are collected from operational IoT networks and thus reflect real-world conditions. Specifically, CIC-ToN-IoT and NF-UQ-NIDS are built from real network traffic in smart environments, and Edge-IIoTset captures edge device behavior in industrial settings. These datasets contain realistic attack patterns and traffic flows observed in actual deployments, making them well-suited for evaluating practical intrusion detection capabilities.

To apply ADAP-GNN to these datasets, the graph properties are analyzed. Namely, the proportion of attackers and degree centrality entropy are computed for each dataset. The values are presented in Table 2 alongside the corresponding ADAP-GNN configuration decisions. The table illustrates that datasets with a high proportion of attackers, such as Edge-IIoTset with 77.6% of nodes as attackers, should utilize the neighbor sampling technique to efficiently manage computational overhead. When the proportion of attackers is high, it is safe to use a subset of the neighborhood while aggregating information. A dataset with a low proportion of attackers should not use neighbor sampling. For instance, the CIC-IDS-2017 dataset has only 0.052% of the nodes as attackers; for that reason, there is a high risk in sampling neighboring nodes. To utilize the attention mechanism, the degree centrality entropy should be high. As the values are normalized between zero and one, degree centrality entropy of value 0.022 is considered low, which indicates that the Edge-IIoTset dataset should use uniform weights while aggregating information from neighboring nodes, not the attention mechanism. On the other hand, the X-IIoTID dataset has a 0.829 degree centrality entropy, which is high, indicating that an attention mechanism should be used in this dataset.

4.2. Experimental setup

We implemented our GNN models using PyTorch and the Deep Graph Library (DGL). Each dataset was partitioned into three parts: training, validation, and testing. After each epoch, the model are tested on the validation set to check for overfitting. As a checkpoint, the model with the best F1 score on the validation set is used; this model was then tested on the testing set. The

Table 3

Comparison of F1-scores among traditional deep learning models (MLP, CNN, GRU, LSTM) and graph-based approaches (E-GraphSAGE, E-GAT) across multiple intrusion detection datasets. Bold values indicate the best performance for each dataset.

Dataset	MLP	CNN	GRU	LSTM	E-GraphSAGE	E-GAT
CIC-IDS-2017	92.11	97.11	95.39	95.19	98.32	98.58
CIC-ToN-IoT	77.05	79.68	76.72	76.53	84.85	84.61
CIC-ToN-IoT-Snapshot	76.56	78.68	75.36	76.29	85.04	84.63
NF-UQ-NIDS	81.68	85.61	85.78	85.49	87.22	84.81
NF-UQ-NIDS-Unified-Community	87.43	87.39	83.25	84.22	87.77	90.78
NF-CSE-CIC-IDS2018	84.12	91.32	89.56	90.45	95.79	95.41
X-IIoTID	87.25	89.35	90.27	89.86	92.14	94.77
CCD-INID-Centrality-Focused	80.43	83.51	82.41	82.65	85.32	87.47
Edge-IIoTset	98.09	99.12	98.12	98.01	95.91	95.21

number of training epochs was adjusted based on dataset size, ranging from 500 for larger datasets to 5000 for the smallest one. This selection was made to balance computational efficiency with convergence, making sure to not stop training unless further reduction in loss became negligible. The learning rate was set to 0.005 for the Adam optimizer. To address the class imbalance present in several datasets, we used class-weighted cross-entropy loss, where weights are inversely proportional to the class frequencies. This adjustment ensures that minority attack types contribute proportionally during training, mitigating bias toward dominant classes. We set the number of GNN layers to 2, with 25 and 10 neighbors sampled for aggregation in the first and second layers, respectively. The final embedding dimension for nodes was set to 128, and a ReLU activation function was used. To reduce the risk of overfitting, we applied dropout with a probability of 0.2 and weight decay with a coefficient of 10^{-3} . For a fair evaluation and comparison, we used the weighted F1-score as a key metric.

To ensure robust and stable evaluation, each model configuration was trained multiple times using different random seeds. At the end of training, the five checkpoints with the highest validation F1-scores were retained. These were then tested independently on the test set, and the maximum test score among them was reported. This strategy minimizes the impact of random initialization or noisy validation peaks and reflects the realistic deployment scenario where the best model from training is selected. Moreover, we observed that performance trends across datasets and configurations remained consistent across different runs.

4.3. Results

This section presents the experimental outcomes of evaluating E-GraphSAGE and E-GAT in detecting intrusions, and how the ADAP-GNN works in datasets with different graph properties. It is organized into three main parts. First, the performance of E-GraphSAGE and E-GAT is compared with traditional deep learning models, highlighting the effective use of relational patterns in network traffic. Second, the impact of neighbor sampling is analyzed, focusing on how variations in attacker proportions influence the aggregation process. A time-comparison analysis will be added to highlight the advantages of neighbor sampling. Third, the effect of the attention mechanism is highlighted in relation to degree centrality entropy, identifying cases where model performance is enhanced by using the attention mechanism. To add practical analysis, a comparison of time needed to classify records is presented.

4.3.1. Performance analysis of E-GraphSAGE and E-GAT in NIDS

Graph Neural Networks are designed to explore relational patterns in graphs. Using these complex patterns between devices, GNNs are a powerful tool in NIDS environments. To demonstrate this effectiveness, the performance of GNNs is compared with previous deep learning models used in previous works: Multi-Layer Perceptron (MLP), Convolutional Neural Network (CNN), Gated Recurrent Units (GRU), and Long Short-Term Memory (LSTM). MLP is a basic feedforward network that learns global patterns without explicitly modeling spatial or temporal structures. CNN applies convolutional filters to capture complex spatial patterns before passing the information to feedforward layers. GRU and LSTM, both variants of Recurrent Neural Networks (RNN), leverage temporal dependencies in sequential data. In the experiments, the same data pre-processing, data partitioning, and experimental settings are used to make a fair comparison. Final F1-scores of all models on each dataset are presented in Table 3. The results indicate that while convolutional and recurrent models benefit from additional information and extract better patterns, models incorporating graph-based representations achieve superior performance across the majority of datasets. This highlights the importance of Graph Neural Networks in advancing Network Intrusion Detection Systems. However, no clear winner is visible when comparing E-GraphSAGE and E-GAT, highlighting the need for a selection algorithm that helps determine which GNN architecture to use.

To highlight the importance of graph-based models, a focused analysis was made on port scanning detection in the CIC-IDS-2017 dataset. Port scanning is the systematic process of looking through open ports on a target system to gather information about its operating system and running applications, often used to identify potential vulnerabilities [37]. Since port scanning typically involves interactions across multiple devices, it is easier to be recognized in a graph context rather than as an isolated record. In such a scenario, GNNs excel as they leverage relational information to capture and analyze the complex patterns formed by these multi-device interactions. Fig. 6 presents the performance of deep learning models alongside GNN models at classifying the PortScan attacks in particular. The results demonstrate that GNN-based models consistently achieve higher F1-scores compared

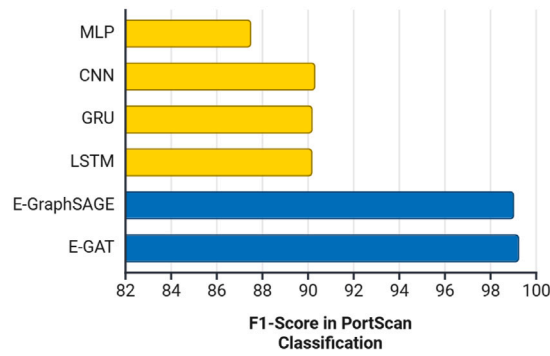


Fig. 6. F1-scores for port scan detection in the CIC-IDS-2017. The yellow bars represent the F1-scores of deep learning models (MLP, CNN, GRU, and LSTM), while the blue bars represent that of graph-based models (E-GraphSAGE and E-GAT). The results clearly demonstrate that graph-based approaches significantly outperform conventional models in detecting port scanning attacks.

Table 4

GNN performance across the datasets sorted based on the proportion of attackers metric. The table compares the F1-score of models using aggregation from the full neighbor and from the neighbor sampling approach. Bolded values indicates the highest accuracy on that dataset. Results demonstrate that neighbor sampling is better at higher proportions of attackers.

Dataset	Proportion of attackers	Full neighborhood	Sampled neighborhood
CIC-ToN-IoT	0.000077	84.85	82.09
CIC-ToN-IoT-Snapshot	0.000122	85.04	82.34
CIC-IDS-2017	0.000523	98.32	98.13
NF-CSE-CIC-IDS2018	0.007862	95.79	94.46
NF-UQ-NIDS	0.007998	87.22	85.84
NF-UQ-NIDS-Unified-Community	0.073620	87.77	91.03
X-IIoTID	0.080357	92.14	91.64
CCD-INID-Centrality-Focused	0.475728	85.32	85.55
Edge-IIoTset	0.776236	95.91	96.41

to traditional deep learning in detecting this type of attacks. MLP achieved 87.54, CNN 90.35, GRU 90.23, and LSTM 90.22. In contrast, graph-based models delivered significantly higher performance with GraphSAGE reaching 99.06 and GAT 99.28. These findings show the importance of incorporating graphical information to enhance network intrusion detection systems.

4.3.2. Importance of neighbor sampling in NIDS

Building on the superior results of Graph Neural Networks (GNNs) over traditional deep learning models in NIDS, the impact of various GNN model configurations on the performance is studied. Neighbor sampling selects a subset of the set of neighboring nodes randomly instead of aggregating information from the entire set. This reduces computational complexity, improves time efficiency, and serves as a regularizer that prevents overfitting. However, it risks neglecting critical information if the information about attacks is rare. For that reason, neighbor sampling is not recommended by ADAP-GNN if the proportion of attackers is low in the graph. To highlight the effect of the proportion of attackers on the choice of using neighbor sampling in the aggregation step, experiments were held across datasets with varying proportions of attackers.

To highlight the effect of the proportion of attackers on the choice of neighbor sampling in the aggregation step, the scores of the models are listed alongside the proportions in Table 4. The values in bold represent the higher F1-score between the two configurations of the model. Our results indicate that full neighbor aggregation tends to perform better for networks with a low proportion of attackers. Conversely, when the proportion of attackers is higher, neighbor sampling yields superior performance. The increase in scores for models highlights the generalization ability added by the neighbor sampling technique. This supports the claims we made in the property-aware selection algorithm. For instance, in datasets with a very low proportion of attackers, such as CIC-ToN-IoT (0.000077) and CIC-IDS-2017 (0.000523), full neighborhood aggregation achieved F1-scores of 84.85 and 98.32, respectively, compared to 82.09 and 98.13 using sampling. Conversely, in datasets with higher attacker proportions, such as NF-UQ-NIDS-Unified-Community (0.073620) and Edge-IIoTset (0.776236), the sampled approach outperformed the full aggregation, achieving 91.03 and 96.41 versus 87.77 and 95.91.

To determine when neighbor sampling should be used according to the Proportion of Attackers metric, a threshold is needed. To select this threshold, we employed logistic regression using the results from Table 4. Each dataset was labeled with a binary outcome indicating whether neighbor sampling outperformed full aggregation. We then fit a logistic function to model the probability that neighbor sampling should be selected, as a function of the proportion of attackers in the graph. The decision threshold was chosen

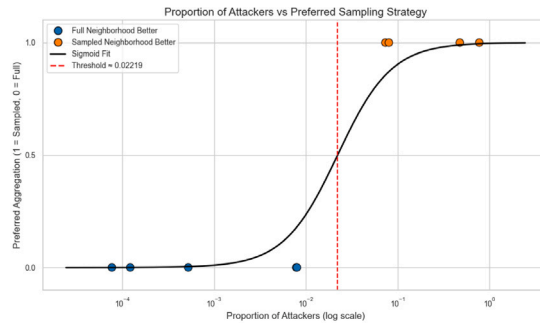


Fig. 7. Logistic regression used to determine the threshold on the proportion of attackers. The decision threshold $p^* = 0.022$ corresponds to the inflection point of the fitted sigmoid curve.

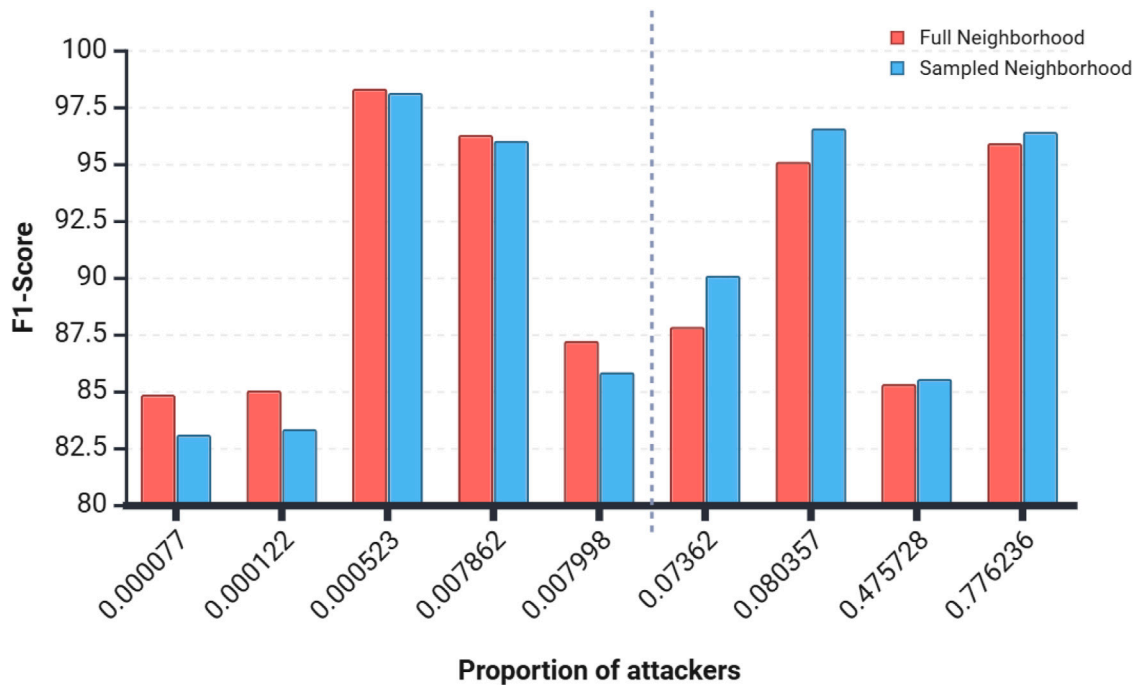


Fig. 8. Bar graph comparing the F1 scores of E-GraphSAGE across the datasets while using full neighborhood and sample neighborhood for aggregation. The x-axis displays the proportion of attackers in the datasets arranged in ascending order. A vertical dashed line is included as a visual indicator to show where the performance advantage shifts in favor of the neighbor sampling.

at the inflection point of the sigmoid curve, which corresponds to the point where the model predicts a 50% chance of favoring sampling. This analysis yielded a threshold of approximately $p^* = 0.022$. Therefore, ADAP-GNN recommends enabling neighbor sampling only when the proportion of attackers exceeds this threshold. The fitted curve and threshold point are shown in Fig. 7, which supports the property-aware design of our selection mechanism.

Fig. 8 offers a visual overview of how the proportion of attackers influences the performance of different neighborhood aggregation strategies. The bar graph shows how the aggregation of edges from the full neighborhood of a node (red bars) is better for lower proportions of attackers. Sampling in a network with few attackers may risk missing important information, which necessitates the usage of all available edges and nodes. Conversely, as the proportion of attackers increases, the sampled neighborhood approach (blue bars) outperforms. The increased diversity of classes available allows a stochastic neighbor sampling technique to be applied.

Our results show how closely linked the GNN configurations are to the graph properties. The core of a GNN model is the aggregation step, where a node aggregates information from all incoming edges from neighboring nodes. Choosing a random sample of the neighborhood of a node can decrease the time by 50% as shown in Fig. 9, an improvement needed for rapid classification of new threats. Additionally, sampling acts as a regularization that masks some data to improve the generalizability of the model. However, this risks losing important data if some classes are concentrated on a small number of nodes that can be all sampled out. For that reason, the proportion of attackers in the networks plays a great role in choosing whether to sample or not. The difference

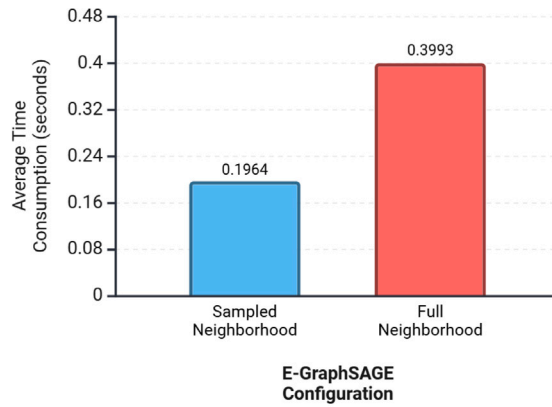


Fig. 9. Average testing time in seconds of E-GraphSAGE model configuration across datasets. The blue bar represents the time needed for the E-GraphSAGE model using a sampled neighborhood at the aggregation phase to classify all records in the testing set, while the red bar represents that of the model using a full neighborhood. The difference demonstrates the gain in time when using neighbor sampling.

Table 5

List of datasets sorted by the degree centrality entropy, comparing the performance (F1-score) of the model with and without attention mechanism. Bolded values indicates the highest accuracy on that dataset. Results demonstrate that attention mechanism is less prone to overfit at higher values of degree centrality entropy.

Dataset	Degree centrality entropy	Without attention	Attention
Edge-IIoTset	0.021798	95.91	95.21
CIC-ToN-IoT	0.174882	84.85	84.61
CIC-ToN-IoT-Snapshot	0.176466	85.04	84.63
NF-CSE-CIC-IDS2018	0.239182	95.79	95.41
NF-UQ-NIDS	0.240163	87.22	84.81
CIC-IDS-2017	0.440548	98.32	98.58
NF-UQ-NIDS-Unified-Community	0.687675	87.77	90.78
CCD-INID-Centrality-Focused	0.814577	85.32	87.47
X-IIoTID	0.829736	92.14	94.77

in performances highlights that there is no one configuration that works all the time and that a property-aware selection algorithm is needed, proving the importance of applying ADAP-GNN in NIDS to create robust GNN classifiers.

In addition to performance benefits, we observed that neighbor sampling reduced computational time and memory usage, particularly in datasets with high graph density. This is because dense graphs significantly increase the number of neighbors to aggregate, making full aggregation costly. For example, in NF-UQ-NIDS, neighbor sampling reduced training time by over 63% and memory usage by more than 300 MB. These trends, while secondary to our primary goal of adaptive model selection, highlight an important computational advantage when operating under graph-aware configurations.

4.3.3. Importance of attention mechanism in NIDS

In addition to neighbor sampling, ADAP-GNN determines whether to utilize the attention mechanism based on degree centrality entropy. This measure represents how uniformly node connectivity is distributed. A low entropy value suggests that connectivity is concentrated in a few nodes, potentially leading to overfitting when using attention. Conversely, higher entropy indicates a more balanced network structure where attention can effectively highlight relevant features.

To highlight the effect of the degree centrality entropy on the choice of using an attention mechanism or not, a comparison of the performance of the GNN models is presented in Table 4. The results are sorted by the degree centrality entropy metric. For each dataset, the values in bold represent the higher F1-score between the two configurations of the model. The results indicate that the E-GAT model performs better than E-GraphSAGE in datasets with higher values of degree centrality entropy. Conversely, when the degree centrality entropy is low, E-GraphSAGE is the choice. For example, the CIC-IDS-2017 dataset, with an entropy of 0.440548, shows an F1-score improvement from 98.32 (without attention) to 98.58 (with attention). Similarly, in datasets like NF-UQ-NIDS-Unified-Community (entropy 0.687675) and X-IIoTID (entropy 0.829736), the attention mechanism yields superior performance, whereas in datasets with very low entropy such as Edge-IIoTset (entropy 0.021798), the model without attention performs better (95.91 vs. 95.21).

To systematically decide when the attention mechanism should be applied, we used logistic regression to capture the relationship between the degree centrality entropy and the relative benefit of using attention. Each dataset was labeled according to whether

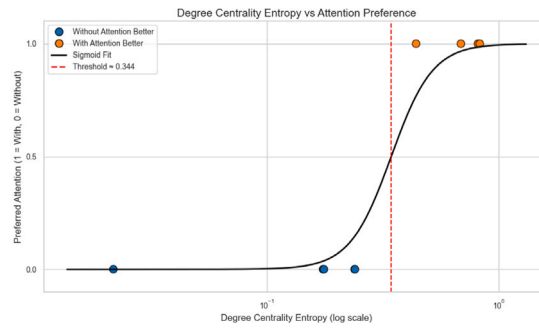


Fig. 10. Logistic regression to determine the threshold on degree centrality entropy. The attention mechanism is recommended when the entropy exceeds $E^* = 0.344$.

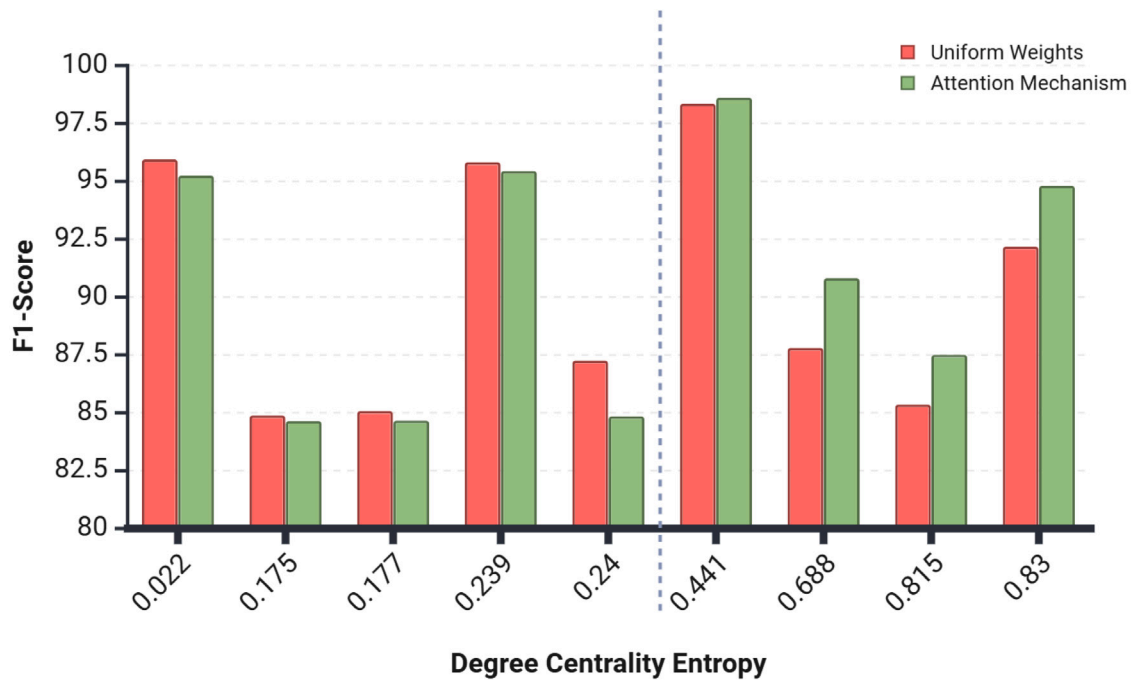


Fig. 11. Bar graph comparing the F1-scores of E-GraphSAGE configured with an attention mechanism versus uniform weighting across datasets. The x-axis displays the degree centrality entropy of the datasets arranged in ascending order. A vertical dashed line is included as a visual cue to indicate where the performance advantage shifts in favor of the attention mechanism.

E-GAT outperformed E-GraphSAGE in terms of weighted F1-score. By fitting a sigmoid function to this binary outcome, we chose the threshold at inflection point at which attention begins to consistently improve performance. The threshold value was approximately $E^* = 0.344$, which we adopt in ADAP-GNN to guide the use of attention. As shown in Fig. 10, this value reflects the boundary where attention would be effective without inducing overfitting. This threshold supports the rule-based mechanism integrated in our model selection strategy.

Fig. 11 offers a visual overview of how the degree centrality entropy affects the performance of different mechanisms to weight the contribution of neighboring nodes. The results are sorted by the degree centrality entropy of the graph constructed for each dataset. The red bars represent the F1-score of the models using uniform weighting aggregation, while the green bars represent those of the models using attention mechanism. The bar graph highlights how the results of both configurations are similar, with a small advantage of uniform weighting aggregation in low-entropy environments. However, the inverse is true with higher degree centrality entropy values, as shown by the higher green bars of the models utilizing the attention mechanism. Only one exception can be spotted in the NF-UQ-NIDS dataset, which has a 0.24-degree centrality entropy. The uniform weighting aggregation significantly outperforms the attention mechanism, a case that should be discussed in detail.

In theory, using an attention mechanism should yield better results since it is guiding the model to choose more relevant data. However, in the case of NF-UQ-NIDS dataset uniform weighting outperforms it. The main reason for this is overfitting on

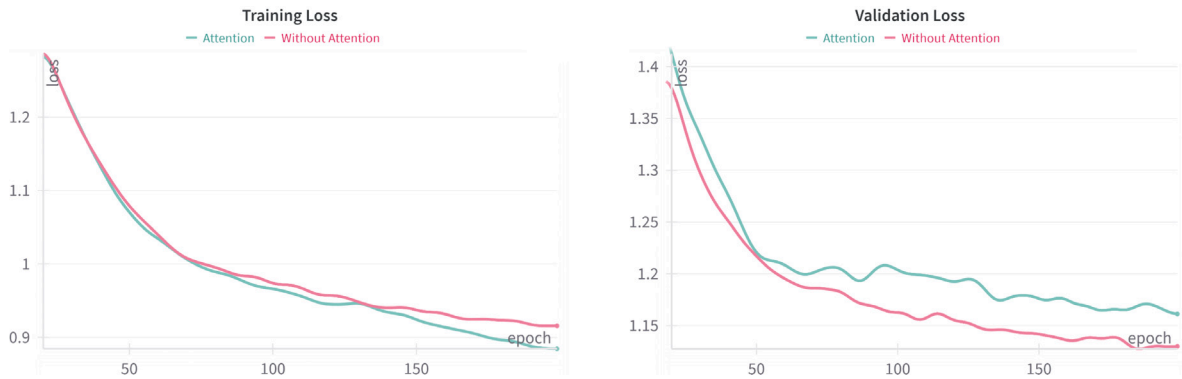


Fig. 12. Training and validation loss curves for GNN configurations with and without attention mechanism on the NF-UQ-NIDS dataset. Comparing the figures, an overfitting can be spotted using the attention mechanism.

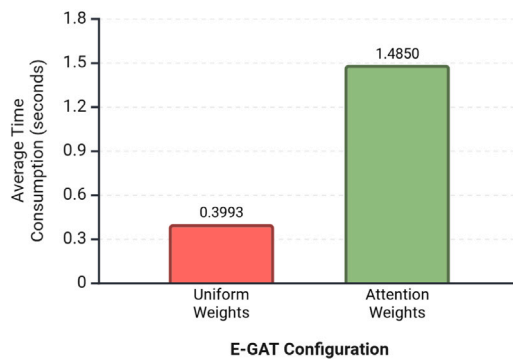


Fig. 13. Average testing time in seconds of E-GAT model configuration across datasets. The red bar represents the time needed to classify all records in the testing set by the E-GAT model using a uniform weighting, while the green bar represents that of the model using a attention mechanism. The difference demonstrates the computational overhead when using attention mechanism.

complex graphical patterns in training sets. Fig. 12 presents two line graphs showing the loss values over epochs for the two GNN configurations, original aggregation and using an attention mechanism to weight neighbors. On the training set, the loss reaches a lower level, indicating a higher convergence during training. However, on the validation set, the same model has a higher loss compared to the other configuration. This suggests a tendency toward overfitting. In our experiments, different values of dropout and weight decay are used to counter the overfitting, and all show the same trend of overfitting when using the attention mechanism. This is further reflected in the test results, where a higher F1-score was achieved by the original configuration without attention (87.22%) as shown in Table 5. For that reason, attention mechanism should be used only in high-entropy datasets.

Our findings demonstrate the strong correlation between the graph features and the GNN configurations. The aggregation step, in which a node compiles data from all incoming edges from nearby nodes, is the key component of a GNN model. Originally, a GNN model treats all nodes equally, where each neighbor can have the same degree of effect on a node. To improve this, an attention mechanism can be used to identify neighbors with the most relevant information and assign to them higher weights, guiding the model to uncover better patterns. However, calculating dynamic weights introduces additional overhead, which should be considered in the NIDS context. The time required by the models to run on the whole testing graph was recorded throughout the experiments, and the average is presented in Fig. 13. For attention-based models, the primary factor influencing time and memory was the average degree across all nodes in the graph rather than graph density. Datasets with low average degree, such as CIC-ToN-IoT (75.01) and Edge-IIoTse (22.99), have a shorter inference time compared to CIC-IDS-2017 (294.3) and NF-UQ-NIDS (229.14). This is due to the fact that attention weights should be calculated between the source node and all its neighbors, and a high degree average means more weights should be calculated. These findings highlight that scalability is highly tied to structural properties of the graph, particularly the average degree in the graph. Although attention introduces additional overhead due to the calculation of dynamic weights, the incremental processing time remains modest when offset by the performance improvements in high-entropy datasets. This underscores the adaptive strategy of ADAP-GNN: employing attention only when its accuracy improvements justify the additional computational cost.

In summary, our study demonstrates how effective the GNN-based approaches are to detect intrusions and threats. By using ADAP-GNN, a novel light-weight interpretable rule-based selection algorithm, we were able to adapt the configuration of the GNN-based NIDS to fit the type of network they will be applied in. Our approach explains how the GNN interacts with the graph and aggregates information. This paves the road for future works that explore more configurations and provide adaptive, efficient, and practical frameworks to apply the most accurate NIDS in real-world environments.

While the proposed ADAP-GNN framework demonstrates promising results across diverse NIDS datasets, several limitations remain that open avenues for future research. First, our rule-based selection algorithm relies on two graph-level metrics and fixed thresholds calibrated on nine datasets; additional metrics may further refine the selection algorithm. Second, we evaluated models on pre-collected flow datasets. To deploy the models in dynamic real-world IoT networks, a sliding-window would be necessary. Third, we addressed class imbalance using weighted F1-scores and class-weighted loss functions, but did not explore other techniques such as data augmentation for rare attack types. Fourth, while our model selection reduces inference time and we discussed time complexity, a comprehensive analysis of memory usage, model scalability, and deployment constraints on edge devices is beyond the scope of this work. Finally, although ADAP-GNN was not explicitly designed to counter adversarial or poisoning attacks, its selection logic discourages architectures prone to overfitting, especially in graphs with uniform structural characteristics. In particular, attention-based models are automatically disabled in low-entropy graphs, where they risk amplifying the influence of dominant nodes. This mitigates certain overfitting patterns typically exploited by adversarial flows. Moreover, regularization techniques such as dropout and weight decay were used in training to improve generalization. Future extensions of this work may consider adversarial training, attention smoothing, or robust message-passing mechanisms to further strengthen resilience in hostile environments.

5. Conclusion

This study introduced ADAP-GNN, a lightweight and interpretable adaptive property-aware framework to enhance Network Intrusion Detection Systems in IoT environments. Our approach leverages domain-specific insights to select the most appropriate GNN configuration based on graph-derived metrics that reflect both the structural and security characteristics of the IoT network, enabling an informed model selection. Our first contribution addresses the limitations of standard node-centric GNN architectures in NIDS settings by developing edge-centric variants, including a novel E-GAT model. This architecture reformulates the attention mechanism to operate over edge features. Alongside this, we extended the existing E-GraphSAGE model by incorporating neighbor sampling, enabling scalable learning in large graphs. Our second contribution lies in the design of a rule-based selection algorithm that chooses between E-GAT and E-GraphSAGE configurations based on pre-computed graph metrics. Specifically, the proportion of attackers is used to specify whether neighbor sampling should be enabled or not, and the degree centrality entropy is used to determine whether an attention mechanism is beneficial. Experimental results on nine diverse NIDS datasets confirm that our framework makes robust decisions: in low-attacker-density graphs, full neighborhood aggregation ensures coverage of rare malicious patterns, while neighbor sampling becomes advantageous in high-attacker-density environments, reducing inference time without performance loss. Similarly, E-GAT's attention mechanism is better in high-entropy graphs by prioritizing informative connections, whereas uniform weighting outperforms in low-entropy settings by reducing overfitting. Our experiments reveal that full neighborhood aggregation is most effective in scenarios with a low proportion of attackers, ensuring that essential information is comprehensively captured. In contrast, when attacker density is higher, employing neighbor sampling not only acts as a regularizer to improve generalization but also significantly reduces computational time by up to 50%. Furthermore, the incorporation of an attention mechanism proves particularly beneficial in high-entropy graphs by emphasizing relevant features, which increases the F1-score significantly. On the other hand, uniform weighting in low-entropy settings helps mitigate the risk of overfitting and increase time efficiency. Overall, the proposed ADAP-GNN framework provides a principled, explainable, and deployable strategy that balances between classification accuracy and computational efficiency. This light-weight interpretable property-aware framework offers a robust solution for intrusion detection in resource-constrained IoT environments. Looking ahead, our future works will explore the incorporation of additional graph metrics and alternative GNN architectures into the rule-based selection framework. Moreover, we plan to validate our approach in real-world operational environments to further assess its scalability, robustness, and adaptability in the face of evolving cyber threats.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by CESI EAST Region and Ektidar, a Lebanese project aimed at empowering youth.

Data availability

Code is available on Github, and link is in the footnotes in the paper.

References

- [1] Samantaray M, Barik RC, Biswal AK. A comparative assessment of machine learning algorithms in the IoT-based network intrusion detection systems. *Decis Anal J* 2024;11:100478.
- [2] Abdulganiyu OH, Tchakouch TA, Saheed YK. Towards an efficient model for network intrusion detection system (IDS): systematic literature review. *Wirel Netw* 2024;30(1):453–82.
- [3] Nandanwar H, Kataraya R. Deep learning enabled intrusion detection system for Industrial IOT environment. *Expert Syst Appl* 2024;249:123808.
- [4] Zhong M, Lin M, Zhang C, Xu Z. A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges. *Comput Secur* 2024;141:103821.
- [5] Bilot T, Madhoun NE, Agha KA, Zouaoui A. Graph neural networks for intrusion detection: A survey. *IEEE Access* 2023;11:49114–39. <http://dx.doi.org/10.1109/ACCESS.2023.3275789>.
- [6] Wu Z, Pan S, Chen F, Long G, Zhang C, Yu PS. A comprehensive survey on graph neural networks. *IEEE Trans Neural Networks Learn Syst* 2021;32(1):4–24. <http://dx.doi.org/10.1109/TNNLS.2020.2978386>.
- [7] Li Y, Zhou Z, Li R, Shi F, Guo J, Liu Q. GoGDDoS: A multi-classifier for DDoS attacks using graph neural networks. In: 2023 IEEE symposium on computers and communications. ISCC, 2023, p. 1462–7. <http://dx.doi.org/10.1109/ISCC58397.2023.10218316>.
- [8] Hamilton WL, Ying R, Leskovec J. Inductive representation learning on large graphs. In: *Proceedings of the 31st international conference on neural information processing systems*. NIPS '17, Red Hook, NY, USA: Curran Associates Inc.; 2017, p. 1025–35.
- [9] Khemani B, Patil S, Kotecha K, Tanwar S. A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions. *J Big Data* 2024;11(1):18.
- [10] Lo WW, Layeghy S, Sarhan M, Gallagher M, Portmann M. E-GraphSAGE: A graph neural network based intrusion detection system for IoT. In: *NOMS 2022-2022 IEEE/IFIP network operations and management symposium*. 2022, p. 1–9. <http://dx.doi.org/10.1109/NOMS54207.2022.9789878>.
- [11] Abdulkareem SA, Heng Foh C, Shojafar M, Carrez F, Moessner K. Network intrusion detection: An IoT and non IoT-related survey. *IEEE Access* 2024;12:147167–91. <http://dx.doi.org/10.1109/ACCESS.2024.3473289>.
- [12] Sowmya T, Mary Anita E. A comprehensive review of AI based intrusion detection system. *Meas: Sensors* 2023;28:100827. <http://dx.doi.org/10.1016/j.measen.2023.100827>.
- [13] Park SB, Jo HJ, Lee DH. G-IDCS: Graph-based intrusion detection and classification system for CAN protocol. *IEEE Access* 2023;11:39213–27. <http://dx.doi.org/10.1109/ACCESS.2023.3268519>.
- [14] Ghalmane Z, Hassouni ME, Cherifi H. Immunization of networks with non-overlapping community structure. *Soc Netw Anal Min* 2019;9(1):45.
- [15] Ghalmane Z, Cherifi C, Cherifi H, Hassouni ME. Extracting modular-based backbones in weighted networks. *Inform Sci* 2021;576:454–74.
- [16] Termos M, Ghalmane Z, Brahmia M-E-A, Fadlallah A, Jaber A, Zghal M. Intrusion detection system for IoT based on complex networks and machine learning. In: 2023 IEEE intl conf on dependable, autonomic and secure computing. DASC, 2023, p. 0471–7. <http://dx.doi.org/10.1109/DASC/PiCom/CBDCCom/Cy59711.2023.10361433>.
- [17] Termos M, Ghalmane Z, el Amine Brahmia M, Fadlallah A, Jaber A, Zghal M. GDLC: A new graph deep learning framework based on centrality measures for intrusion detection in IoT networks. *Internet Things* 2024;26:101214. <http://dx.doi.org/10.1016/j.iot.2024.101214>.
- [18] Waikhom L, Patgiri R. A survey of graph neural networks in various learning paradigms: methods, applications, and challenges. *Artif Intell Rev* 2023;56(7):6295–364. <http://dx.doi.org/10.1007/s10462-022-10321-2>.
- [19] Pujol-Perich D, Suarez-Varela J, Cabellos-Aparicio A, Barlet-Ros P. Unveiling the potential of graph neural networks for robust intrusion detection. *SIGMETRICS Perform Eval Rev* 2022;49(4):111–7. <http://dx.doi.org/10.1145/3543146.3543171>.
- [20] Zhang Y, Jue C, Liu W, Ma Y. GRAN: a SDN intrusion detection model based on graph attention network and residual learning. *Comput J* 2024;bxae108. <http://dx.doi.org/10.1093/comjnl/bxae108>.
- [21] Liang F, Qian C, Yu W, Griffith D, Golmie N. Survey of graph neural networks and applications. *Wirel Commun Mob Comput* 2022;2022(1):9261537.
- [22] Veličković P, Cucurull G, Casanova A, Romero A, Liò P, Bengio Y. Graph attention networks. In: *International conference on learning representations*. 2018.
- [23] Gorricho-Segura M, Echeberria-Barrio X, Seguro-Gil L. Edge-based analysis for network intrusion detection using a GNN approach. In: 2023 JNIC cybersecurity conference. JNIC, 2023, p. 1–7. <http://dx.doi.org/10.23919/JNIC58574.2023.10205384>.
- [24] Gao Y, Yang H, Zhang P, Zhou C. GraphNAS: Graph neural architecture search with reinforcement learning. 2019.
- [25] Zhou K, Huang X, Song Q, Chen R, Hu X. Auto-GNN: Neural architecture search of graph neural networks. *Front Big Data* 2022;Volume 5 - 2022. <http://dx.doi.org/10.3389/fdata.2022.1029307>.
- [26] Sarhan M, Layeghy S, Portmann M. Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection. *Big Data Res* 2022;30:100359. <http://dx.doi.org/10.1016/j.bdr.2022.100359>.
- [27] Al Tfaily F, Ghalmane Z, Termos M, Brahmia M-E-A, Jaber A, Zghal M. Generating realistic cyber security datasets for IoT networks with diverse complex network properties. In: *Proceedings of the 10th international conference on internet of things, big data and security - IoTBDS*. 2025.
- [28] Sharafaldin I, Lashkari AH, Ghorbani AA. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* 2018;1:108–16.
- [29] Sarhan M, Layeghy S, Moustafa N, Portmann M. NetFlow datasets for machine learning-based network intrusion detection systems. In: Deze Z, Huang H, Hou R, Rho S, Chilamkurti N, editors. *Big data technologies and applications*. Cham: Springer International Publishing; 2021, p. 117–35.
- [30] Liu Z, Thapa N, Shaver A, Roy K, Siddula M, Yuan X, Yu A. Using embedded feature selection and CNN for classification on CCD-INID-V1-A new IoT dataset. *Sensors (Basel)* 2021;21(14).
- [31] Ferrag MA, Friha O, Hamouda D, Maglaras L, Janicke H. Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications: Centralized and federated learning. 2022. <http://dx.doi.org/10.21227/mbc1-1h68>.
- [32] Al-Hawawreh M, Sitnikova E, Aboutorab N. X-IIoTID: A connectivity-agnostic and device-agnostic intrusion data set for industrial internet of things. *IEEE Internet Things J* 2022;9(5):3962–77. <http://dx.doi.org/10.1109/JIOT.2021.3102056>.
- [33] Liu X, Yan M, Deng L, Li G, Ye X, Fan D. Sampling methods for efficient training of graph convolutional networks: A survey. *IEEE/CAA J Autom Sin* 2022;9(2):205–34. <http://dx.doi.org/10.1109/JAS.2021.1004311>.
- [34] Iswarya V, Govindasamy V, Akila V. A survey on the identification of influential spreaders in complex networks. In: 2024 2nd international conference on computer, communication and control. IC4, 2024, p. 1–4. <http://dx.doi.org/10.1109/IC457434.2024.10486364>.
- [35] Ellerman D. *New foundations for information theory: logical entropy and Shannon entropy*. Springer Nature; 2021.
- [36] Ugurlu O. Comparative analysis of centrality measures for identifying critical nodes in complex networks. *J Comput Sci* 2022;62:101738.
- [37] Ono D, Guillen L, Izumi S, Abe T, Suganuma T. A proposal of port scan detection method based on packet-in messages in OpenFlow networks and its evaluation. *Int J Netw Manage* 2021;31(6):e2174. <http://dx.doi.org/10.1002/nem.2174>.